



**MAHATMA EDUCATION SOCIETY'S
PILLAI COLLEGE OF ENGINEERING**

New Panvel, Navi Mumbai

Department of Electronics and Telecommunication Engineering

**Java Programming
PROJECT REPORT**

**To-Do List Application
Java Programming Project**

Submitted By

Name: KRRISH ANKUSH VATANDAR

Roll Number: 567

Class: Third Year EXTC A

Academic Year: 2026–27

Contents

1	Introduction	1
1.1	Objectives	1
2	Problem Statement	3
2.1	Manual Task Management	3
2.2	Difficulty in Tracking Task Status	3
2.3	Task Modification	3
2.4	Task Identification	3
2.5	Data Organization	4
2.6	Input Validation	4
2.7	Need for a Simple User Interface	4
3	Project Overview and Proposed Solution	5
3.1	Project Background	5
3.2	Proposed Solution	5
3.3	Main Features	5
3.4	Application Architecture	6
3.5	Major Components	7
4	Programming Concepts and Design	8
4.1	Overview	8
4.2	Object-Oriented Programming	8
4.3	Classes and Objects	8
4.4	Encapsulation	8
4.5	Methods	8
4.6	ArrayList	9
4.7	Conditional Statements	9
4.8	Loops	9
4.9	Exception Handling	9
4.10	Programming Design Summary	10

5	Working Principle and System Design	11
5.1	Working Principle	11
5.2	Task Lifecycle	11
5.3	Task Data Model	12
5.4	System Workflow	12
6	Technologies Used	13
6.1	Java Programming Language	13
6.2	Java Development Kit	13
6.3	Java Collections Framework	13
6.4	Scanner Class	13
6.5	Exception Handling	13
6.6	File Handling	13
6.7	Integrated Development Environment	14
6.8	Technology Summary	14
7	Performance Analysis	15
7.1	Manual List Versus Java To-Do List Application	15
7.2	Time Complexity Considerations	15
7.3	Memory Usage	16
7.4	User Experience	16
7.5	Performance Summary	16
8	Challenges and Limitations	17
8.1	Data Persistence	17
8.2	Input Errors	17
8.3	Console Interface Limitations	17
8.4	Task Identification	17
8.5	Large Number of Tasks	17
8.6	Date Management	17
8.7	Security	18
8.8	Testing Requirements	18
9	Economic and Practical Impact	19
9.1	Improved Productivity	19
9.2	Reduced Dependence on Paper	19
9.3	Better Task Organization	19
9.4	Educational Value	19
9.5	Extensibility	19

10 Future Scope	21
10.1 Graphical User Interface	21
10.2 Database Integration	21
10.3 User Authentication	21
10.4 Task Reminders	21
10.5 Priority-Based Sorting	21
10.6 Search and Filtering	21
10.7 Cloud Synchronization	22
10.8 Mobile Application	22
10.9 Analytics	22
11 Conclusion	23

List of Figures

3.1	Conceptual architecture of the Java To-Do List Application	6
5.1	Workflow of the To-Do List Application	12

List of Tables

3.1	Major components of the To-Do List Application	7
4.1	Java programming concepts used in the project	10
5.1	Task data model	12
6.1	Technologies used in the To-Do List Application	14
7.1	Comparison of conventional and Java-based task management	15
7.2	Key performance considerations	16
9.1	Practical and educational impact	20

1. Introduction

Time management and task organization are important in academic, professional, and personal activities. People commonly maintain lists of assignments, meetings, deadlines, and daily activities. When these tasks are recorded using unstructured methods such as paper notes, it can become difficult to maintain an accurate and organized list.

A To-Do List Application is a simple software application that allows users to create, view, update, complete, search, and remove tasks from a digital list. The application provides a structured method of managing daily activities and demonstrates fundamental Java programming concepts.

This project is designed as a Java programming project. The main focus is on object-oriented programming, classes and objects, collections, conditional statements, loops, methods, input handling, exception handling, and optional file handling. The application can be implemented as a console-based program and later extended into a graphical or database-driven application.

Each task can contain information such as task ID, title, description, priority, due date, and completion status. A menu-driven interface allows the user to perform the required operations.

1.1 Objectives

The main objectives of the To-Do List Application are:

- To develop a simple task-management application using Java.
- To understand and apply object-oriented programming concepts.
- To create, display, update, complete, search, and delete tasks.
- To maintain multiple tasks using Java collection classes.
- To implement a menu-driven user interface.
- To validate user input and handle common input errors.
- To provide an organized method of managing daily activities.

- To understand how a Java application can be divided into logical modules.
- To provide a foundation for future features such as file storage and databases.

2. Problem Statement

Managing several tasks manually can result in forgotten deadlines, incomplete work, and difficulty in identifying the current status of activities. A digital task-management system can solve these problems by maintaining all tasks in one organized application.

The proposed To-Do List Application provides a centralized list in which each task has an identity and associated information. The user can add a new task, view existing tasks, modify task information, mark a task as completed, search for a task, and remove a task that is no longer required.

2.1 Manual Task Management

A handwritten list becomes difficult to maintain when tasks are frequently added, removed, or modified. A software-based list provides convenient operations for changing the task list.

2.2 Difficulty in Tracking Task Status

Without a clear status field, users may not know whether a task is pending or completed. The application therefore maintains a completion status for every task.

2.3 Task Modification

Task requirements can change after they are created. The application allows the user to update task details such as title, description, priority, or due date.

2.4 Task Identification

When multiple tasks exist, the user needs a simple way to select a particular task. The application uses a task ID to identify individual tasks.

2.5 Data Organization

The application must store several tasks while the program is running. Java collection classes such as `ArrayList` provide a suitable structure for maintaining a dynamic list of task objects.

2.6 Input Validation

Incorrect input such as an invalid task number or an empty task title can cause unexpected behaviour. Input validation is therefore included as an important part of the application.

2.7 Need for a Simple User Interface

A menu-driven console interface makes the application easy to operate while keeping the implementation suitable for a student Java programming project.

3. Project Overview and Proposed Solution

3.1 Project Background

The To-Do List Application is a task-management program developed using Java. It provides the basic operations required to maintain a digital task list. The project is designed with a simple structure so that the relationship between Java programming concepts and application functionality can be clearly understood.

The application starts by displaying a menu. The user selects an operation such as adding a task, viewing tasks, updating a task, marking a task as completed, searching, deleting a task, or exiting the application.

3.2 Proposed Solution

The proposed application uses a **Task** class to represent individual tasks and a task-management component to maintain a collection of these objects. A main class controls program execution and communicates with the user through the console.

The main task information can include:

- Task ID
- Task title
- Task description
- Priority
- Due date
- Completion status

3.3 Main Features

The application provides the following features:

1. Add a new task.

2. Display all tasks.
3. Search for a task.
4. Update task details.
5. Mark a task as completed.
6. Delete a task.
7. Display pending tasks.
8. Display completed tasks.
9. Exit the application.

3.4 Application Architecture

The application can be divided into four logical layers:

1. **User Interface Layer:** Displays the menu and receives user input.
2. **Task Model Layer:** Represents the properties of an individual task.
3. **Task Management Layer:** Performs add, search, update, complete, and delete operations.
4. **Data Layer:** Maintains tasks using an in-memory Java collection and can later be extended to file or database storage.

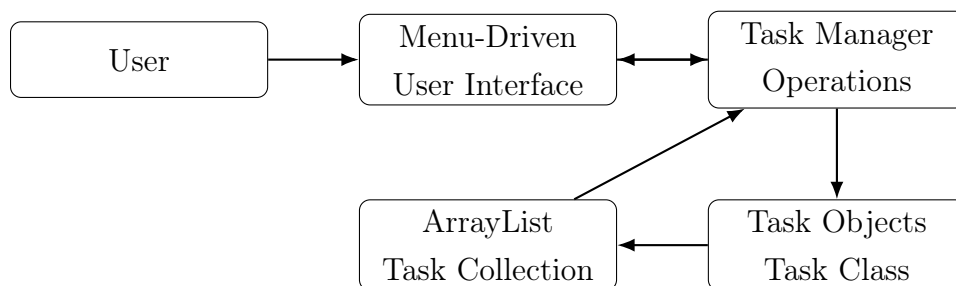


Figure 3.1: Conceptual architecture of the Java To-Do List Application

3.5 Major Components

Table 3.1: Major components of the To-Do List Application

Component	Function
Task class	Represents an individual task and stores its attributes.
Task manager	Performs task operations such as add, update, complete, and delete.
ArrayList	Stores multiple task objects dynamically during execution.
Scanner	Reads user input from the console.
Main class	Starts the program and controls the main application loop.
Menu system	Presents available operations to the user.
Input validation	Checks whether user-entered values are valid.
File storage (optional)	Can save tasks between program executions.

4. Programming Concepts and Design

4.1 Overview

The project demonstrates fundamental Java programming concepts through a practical application. Related information and operations are organized into classes and methods rather than being written as unrelated statements.

4.2 Object-Oriented Programming

Object-oriented programming is central to the project. A task is represented as an object created from a `Task` class. The class contains attributes such as title, description, priority, due date, and completion status.

4.3 Classes and Objects

The `Task` class defines the structure of a task. Individual tasks are objects created from this class. For example, “Complete Java Project” and “Prepare Presentation” can be represented as separate task objects.

4.4 Encapsulation

Task attributes can be declared as private and accessed through public getter and setter methods. This protects the internal task data and provides controlled access.

4.5 Methods

Methods divide the program into manageable operations. Typical methods include:

- `addTask()`
- `viewTasks()`
- `updateTask()`

- `completeTask()`
- `deleteTask()`
- `searchTask()`

4.6 ArrayList

The Java `ArrayList` class can store a dynamic number of task objects. Unlike a fixed-size array, an `ArrayList` can grow or shrink when tasks are added or removed.

4.7 Conditional Statements

Conditional statements such as `if`, `else`, and `switch` are used to make decisions. A `switch` statement can process the menu option selected by the user.

4.8 Loops

A loop keeps the application running until the user chooses the exit option. A `while` loop can repeatedly display the menu, receive input, and execute the selected operation.

4.9 Exception Handling

Java exception handling can manage invalid input and prevent the program from terminating unexpectedly. For example, `try-catch` can handle invalid integer input.

4.10 Programming Design Summary

Table 4.1: Java programming concepts used in the project

Concept	Application in the project
Class	Defines the structure of a task.
Object	Represents an individual task.
Encapsulation	Protects task data using private fields and methods.
Methods	Divide application operations into reusable units.
ArrayList	Stores multiple task objects.
Switch statement	Processes menu selections.
Loops	Keeps the application running until exit.
Exception handling	Handles invalid user input.
Conditional statements	Checks task status and input conditions.

5. Working Principle and System Design

5.1 Working Principle

The working principle of the To-Do List Application can be explained as follows:

1. **Program initialization:** The application starts and creates the required task collection.
2. **Menu display:** The program displays available task-management operations.
3. **User input:** The user selects an operation by entering a menu option.
4. **Operation execution:** The selected method performs the required task operation.
5. **Data update:** The collection is modified when a task is added, updated, completed, or deleted.
6. **Result display:** The application displays the result of the selected operation.
7. **Menu repetition:** The menu is displayed again until the user chooses to exit.

5.2 Task Lifecycle

A task generally follows this lifecycle:

1. A new task is created.
2. The task is assigned an identifier.
3. The task is stored in the task collection.
4. The task remains pending until completed.
5. The user may update the task details.
6. The task can be marked as completed.
7. The task can eventually be deleted.

5.3 Task Data Model

Table 5.1: Task data model

Attribute	Suggested Type	Purpose
Task ID	int	Identifies the task.
Title	String	Stores the short name of the task.
Description	String	Stores additional information.
Priority	String	Indicates High, Medium, or Low priority.
Due date	String	Stores the expected completion date.
Completed	boolean	Indicates whether the task is completed.

5.4 System Workflow

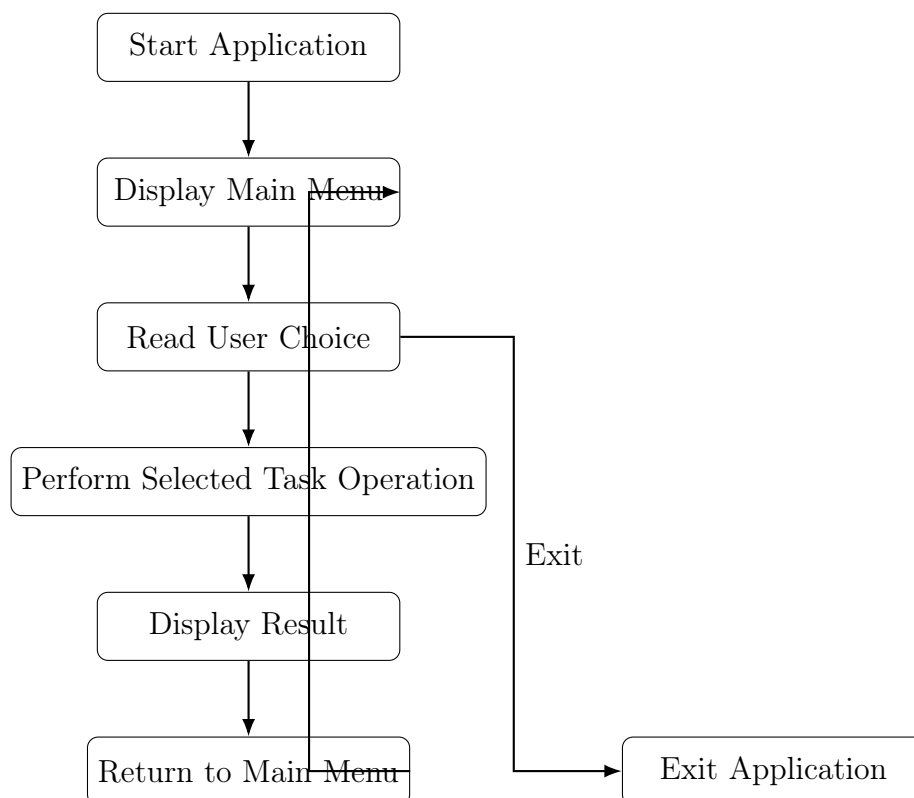


Figure 5.1: Workflow of the To-Do List Application

6. Technologies Used

6.1 Java Programming Language

Java is the primary programming language used to develop the application. Its object-oriented features are suitable for representing tasks as objects and organizing the application into classes and methods.

6.2 Java Development Kit

The Java Development Kit (JDK) provides the compiler and runtime tools required to develop and execute Java programs.

6.3 Java Collections Framework

The Collections Framework provides data structures such as `ArrayList`. The project can use an `ArrayList<Task>` to maintain task objects.

6.4 Scanner Class

The `Scanner` class can receive keyboard input in a console-based implementation.

6.5 Exception Handling

The Java `try-catch` mechanism can handle invalid numeric input and other input-related errors.

6.6 File Handling

The basic application can store tasks only while it is running. File handling can be added using classes such as `File`, `FileWriter`, `FileReader`, or buffered streams to make task data persistent.

6.7 Integrated Development Environment

The project can be developed using IntelliJ IDEA, Eclipse, NetBeans, or the Java command-line tools.

6.8 Technology Summary

Table 6.1: Technologies used in the To-Do List Application

Technology	Application
Java	Main programming language.
JDK	Compilation and execution of Java programs.
OOP	Organization of tasks and application components.
ArrayList	Dynamic storage of task objects.
Scanner	Console input handling.
Exception handling	Management of invalid input.
File I/O	Optional persistent task storage.
IDE	Development, debugging, and execution environment.

7. Performance Analysis

7.1 Manual List Versus Java To-Do List Application

Table 7.1: Comparison of conventional and Java-based task management

Parameter	Manual List	Java Application
Task creation	Written manually	Added through application menu
Task modification	Requires rewriting notes	Update operation modifies stored data
Task deletion	Physical removal or crossing out	Programmatically removed from collection
Status tracking	Manual marking	Boolean completion status
Search	Manual scanning	Search method locates matching tasks
Organization	Depends on user	Structured objects and collection
Validation	Limited	Program can validate input
Scalability	Difficult with many tasks	Collection dynamically stores tasks

7.2 Time Complexity Considerations

For an `ArrayList`-based implementation, adding an item at the end is generally efficient. Searching for a task by ID may require checking multiple elements and can therefore take linear time in the number of tasks. Removing or inserting elements may also require shifting elements.

For a student-level application, an `ArrayList` provides a simple balance between implementation complexity and functionality.

7.3 Memory Usage

The application stores task objects in memory during execution. Memory usage increases with the number of tasks and the amount of text stored in each task.

7.4 User Experience

A clear menu and meaningful messages are important for usability. The user should always know which operation is being performed and whether the operation was successful.

7.5 Performance Summary

Table 7.2: Key performance considerations

Parameter	Importance
Response time	Determines how quickly the application responds to commands.
Search efficiency	Affects the time required to locate a task.
Memory usage	Depends on the number and size of task objects.
Input validation	Improves reliability and prevents invalid operations.
Code modularity	Makes maintenance and future expansion easier.
Data persistence	Determines whether tasks remain after restarting the application.

8. Challenges and Limitations

8.1 Data Persistence

A basic console application using only an `ArrayList` loses task data when the program terminates. File handling or a database is required for permanent storage.

8.2 Input Errors

Users may enter invalid menu choices, task IDs, dates, or empty values. The application must validate input and provide useful error messages.

8.3 Console Interface Limitations

A console interface is simple to implement but does not provide the visual interaction available in a graphical user interface.

8.4 Task Identification

If task IDs are generated incorrectly or reused after deletion, confusion can occur. A suitable ID-generation mechanism should therefore be used.

8.5 Large Number of Tasks

A simple linear search is sufficient for a small project but may become less efficient as the number of tasks becomes very large.

8.6 Date Management

If dates are stored as plain strings, incorrect date formats may be entered. A more advanced implementation can use Java date and time classes for proper validation.

8.7 Security

A local student-level application has limited security requirements. If the application is extended to support user accounts or cloud storage, authentication and data protection would become important.

8.8 Testing Requirements

Each operation should be tested using valid and invalid inputs. Testing should include adding multiple tasks, updating a task, completing a task, deleting a task, searching for existing and non-existing tasks, and exiting the application.

9. Economic and Practical Impact

9.1 Improved Productivity

A digital task list can help users organize activities and reduce the possibility of forgetting important tasks.

9.2 Reduced Dependence on Paper

Replacing handwritten task lists with a digital application makes task management easier to update and reorganize.

9.3 Better Task Organization

Task attributes such as priority, due date, and completion status provide more information than a simple unstructured list.

9.4 Educational Value

The project provides practical experience with Java classes, objects, methods, collections, loops, conditional statements, input handling, and exception handling.

9.5 Extensibility

The basic application can serve as a foundation for more advanced software. Features such as user accounts, reminders, graphical interfaces, databases, and cloud synchronization can be added later.

Table 9.1: Practical and educational impact

Impact Area	Contribution
Productivity	Helps users organize tasks in one application.
Task organization	Provides structured task attributes and status.
Time management	Allows users to maintain due dates and priorities.
Learning	Demonstrates important Java programming concepts.
Maintainability	Encourages modular and object-oriented design.
Extensibility	Provides a foundation for advanced features.

10. Future Scope

10.1 Graphical User Interface

The console application can be converted into a GUI application using JavaFX or Swing. Buttons, forms, tables, and dialog boxes can make task management more user-friendly.

10.2 Database Integration

A relational database such as MySQL can be integrated to permanently store tasks and handle larger amounts of data.

10.3 User Authentication

Multiple users can be supported by adding login and registration functionality. Each user can maintain a separate task list.

10.4 Task Reminders

The application can generate reminders for approaching deadlines and notifications can be implemented using suitable desktop or mobile mechanisms.

10.5 Priority-Based Sorting

Tasks can be automatically sorted according to priority, due date, or completion status.

10.6 Search and Filtering

Advanced filtering can allow users to display only pending, completed, high-priority, or overdue tasks.

10.7 Cloud Synchronization

A future version can connect to a cloud backend so tasks are synchronized across devices.

10.8 Mobile Application

The task-management concept can be extended into an Android application.

10.9 Analytics

A future version can provide statistics such as completed tasks per day, overdue tasks, completion percentage, and productivity trends.

11. Conclusion

The To-Do List Application is a practical Java programming project that demonstrates how basic programming concepts can be combined to solve a real-world organizational problem. The application provides users with a structured way to create, view, update, complete, search, and delete tasks.

The project demonstrates object-oriented programming through classes and objects, while Java collections provide a convenient way to maintain multiple task objects. Methods, loops, conditional statements, switch statements, input handling, and exception handling contribute to the implementation.

The console-based design is suitable for understanding the core functionality of a task-management system. At the same time, the project provides a foundation for future improvements such as file storage, database integration, graphical user interfaces, reminders, authentication, cloud synchronization, and analytics.

Overall, the To-Do List Application demonstrates how Java programming can be used to develop a simple, modular, and extendable software solution for everyday task management.

References

- [1] Oracle, “Java Documentation,” Oracle Java Documentation. Available at: <https://docs.oracle.com/en/java/>
- [2] Oracle, “The Java Collections Framework,” The Java Tutorials. Available at: <https://docs.oracle.com/javase/tutorial/collections/>
- [3] Oracle, “Class ArrayList<E>,” Java Platform SE Documentation. Available at: <https://docs.oracle.com/en/java/javase/>
- [4] Oracle, “Class Scanner,” Java Platform SE Documentation. Available at: <https://docs.oracle.com/en/java/javase/>
- [5] Oracle, “Exceptions,” The Java Tutorials. Available at: <https://docs.oracle.com/javase/tutorial/essential/exceptions/>
- [6] Oracle, “Object-Oriented Programming Concepts,” The Java Tutorials. Available at: <https://docs.oracle.com/javase/tutorial/java/concepts/>